

Basic Information

Experiment No.	03
Date	16/09/26
Title	Digital Payment System using Interface and Custom Exceptions

Objective / Aim

To design and implement a Digital Payment System in Java using an interface, custom checked exceptions, encapsulation, object creation, validation, exception handling, and wallet balance management.

System Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Minimum 1 GHz processor
- At least 500 MB free storage
- Keyboard and mouse

Software Requirements

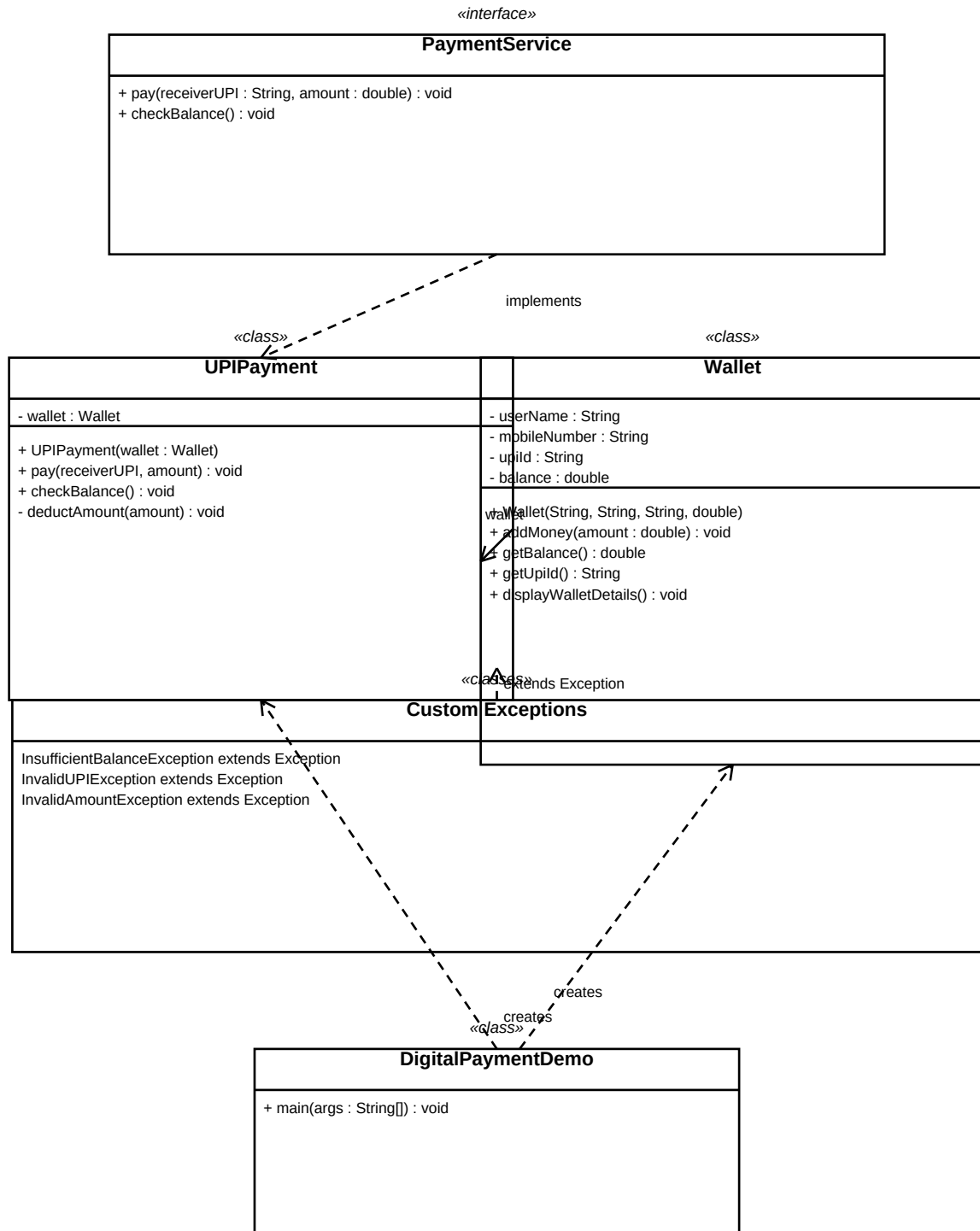
- Windows / Linux / macOS
- JDK 17 or later
- Eclipse / IntelliJ IDEA / NetBeans / VS Code
- Command Prompt / Terminal

Algorithm / Logic Description

1. Define the PaymentService interface with pay() and checkBalance() methods.
2. Create the three custom checked exception classes.
3. Create Wallet with private user details and balance.
4. Initialize wallet details using the constructor.
5. Validate addMoney(); reject zero or negative amounts.
6. Create UPIPayment implementing PaymentService and storing a Wallet object.
7. Validate that the receiver UPI ID contains '@'.
8. Validate that the payment amount is greater than zero.
9. Check available balance and throw InsufficientBalanceException when required.
10. Deduct the payment amount and display the successful transaction.
11. In main(), add Rs.500 and pay Rs.700 to rahul@upi.
12. Handle the three custom exceptions with separate catch blocks and execute finally.
13. Display the final wallet details and balance.

UML Class Diagram

The following is a proper UML-style class diagram matching the supplied Java program. It shows classes, interface, attributes, methods, inheritance, implementation, association, and object-creation relationships.



Source Code

```
// Interface
interface PaymentService {

    // Method to make payment
    void pay(String receiverUPI, double amount)
        throws InvalidUPIException,
               InvalidAmountException,
               InsufficientBalanceException;

    // Method to check balance
    void checkBalance();
}

// Custom Exception 1
class InsufficientBalanceException extends Exception {

    // Constructor
    InsufficientBalanceException(String message) {
        super(message);
    }
}

// Custom Exception 2
class InvalidUPIException extends Exception {

    // Constructor
    InvalidUPIException(String message) {
        super(message);
    }
}

// Custom Exception 3
class InvalidAmountException extends Exception {

    // Constructor
    InvalidAmountException(String message) {
        super(message);
    }
}

// Wallet class
class Wallet {

    // Private data members for encapsulation
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    // Constructor to initialize wallet details
    Wallet(String userName, String mobileNumber,
           String upiId, double balance) {

        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = balance;
    }

    // Method to add money
    void addMoney(double amount) throws InvalidAmountException {

        // Check whether amount is valid
        if (amount <= 0) {
            throw new InvalidAmountException(
                "Invalid amount. Amount must be greater than zero.");
        }

        // Add money to balance
        balance = balance + amount;

        System.out.println("Money added successfully: Rs." + amount);
    }

    // Method to get current balance
    double getBalance() {
        return balance;
    }

    // Method to get UPI ID
    String getUpiId() {
        return upiId;
    }

    // Method to display wallet details
```

```

void displayWalletDetails() {

    System.out.println("\n===== WALLET DETAILS =====");
    System.out.println("User Name      : " + userName);
    System.out.println("Mobile Number: " + mobileNumber);
    System.out.println("UPI ID       : " + upiId);
    System.out.println("Balance      : Rs." + balance);
}
}

// UPIPayment class implements interface
class UPIPayment implements PaymentService {

    // Wallet object
    private Wallet wallet;

    // Constructor
    UPIPayment(Wallet wallet) {
        this.wallet = wallet;
    }

    // Implementing pay() method
    public void pay(String receiverUPI, double amount)
        throws InvalidUPIException,
               InvalidAmountException,
               InsufficientBalanceException {

        // Validate UPI ID
        // UPI ID must contain '@'
        if (receiverUPI == null ||
            !receiverUPI.contains("@")) {

            throw new InvalidUPIException(
                "Invalid UPI ID. Payment cancelled.");
        }

        // Validate payment amount
        if (amount <= 0) {

            throw new InvalidAmountException(
                "Invalid payment amount. Amount must be greater than zero.");
        }

        // Check available balance
        if (amount > wallet.getBalance()) {

            throw new InsufficientBalanceException(
                "Insufficient balance. Payment cancelled.");
        }

        // Deduct amount from wallet
        deductAmount(amount);

        // Display successful transaction
        System.out.println("\nPayment Successful!");
        System.out.println("Receiver UPI : " + receiverUPI);
        System.out.println("Amount Paid  : Rs." + amount);
    }

    // Private method to deduct money
    private void deductAmount(double amount) {

        try {
            // Use reflection to update private balance
            java.lang.reflect.Field field =
                wallet.class.getDeclaredField("balance");

            field.setAccessible(true);

            double currentBalance =
                (double) field.get(wallet);

            field.set(wallet, currentBalance - amount);

        } catch (Exception e) {
            System.out.println("Unable to process payment.");
        }
    }

    // Implementing checkBalance() method
    public void checkBalance() {

        System.out.println(
            "Available Balance: Rs." + wallet.getBalance());
    }
}

// Main class
public class DigitalPaymentDemo {
    public static void main(String[] args) {

```

```

// Create Wallet object
Wallet wallet = new Wallet(
    "Trivikram",
    "9876543210",
    "Trivikram@upi",
    1000.0
);

// Create UPIPayment object
UPIPayment payment = new UPIPayment(wallet);

// Display initial wallet details
wallet.displayWalletDetails();

try {

    // Add money to wallet
    wallet.addMoney(500);

    // Check balance
    payment.checkBalance();

    // Make UPI payment
    payment.pay("rahul@upi", 700);

} catch (InvalidUPIException e) {

    // Handle invalid UPI exception
    System.out.println("Transaction Error: "
        + e.getMessage());

} catch (InvalidAmountException e) {

    // Handle invalid amount exception
    System.out.println("Transaction Error: "
        + e.getMessage());

} catch (InsufficientBalanceException e) {

    // Handle insufficient balance exception
    System.out.println("Transaction Error: "
        + e.getMessage());

} finally {

    // This block always executes
    System.out.println("\nTransaction process completed.");

}

// Display final wallet details
wallet.displayWalletDetails();

// Display final balance
payment.checkBalance();
}
}

```

Compilation and Execution Commands

Save the program as **DigitalPaymentDemo.java**.

```
javac DigitalPaymentDemo.java
java DigitalPaymentDemo
```

The first command compiles the program and the second command executes it.

Output Screenshot

DigitalPaymentDemo - Console Output

```
===== WALLET DETAILS =====
User Name      : Trivikram
Mobile Number: 9876543210
UPI ID         : Trivikram@upi
Balance        : Rs.1000.0
Money added successfully: Rs.500.0
Available Balance: Rs.1500.0

Payment Successful!
Receiver UPI   : rahul@upi
Amount Paid    : Rs.700.0

Transaction process completed.

===== WALLET DETAILS =====
User Name      : Trivikram
Mobile Number: 9876543210
UPI ID         : Trivikram@upi
Balance        : Rs.800.0
```

Final balance: Rs.800.0

Result

The Digital Payment System was successfully implemented using a Java interface and custom exceptions. The program validates transactions, handles exceptions, performs the payment, and displays the updated wallet balance.